

Kovarianz / Kontravarianz

Gegeben seien folgende Subtypbeziehungen:

Kuh SUB→ Tier SUB→ Object Sub SUB→ Super

Anders gesagt: Jede Kuh **ist** ein Tier, jedes Tier **ist** ein Object, jedes SUB **ist** ein Super.

1. Wenn es nun in der Klasse Super eine Methode m() gibt...

Klasse Super Tier m() { ... }

und die Klasse Sub diese Methode überschreiben will, was ist dann rein logisch zulässig?

- a) Klasse Sub Kuh m() { ... } (kovarianter Rückgabetyp)
- b) Klasse Sub Tier m() { ... } (novarianter Rückgabetyp)
- c) Klasse Sub Object m() { ... } (kontravarianter Rückgabetyp)

Da eine Nachricht, die an einen Ausdruck vom Typ Super gesendet wird, aufgrund von Polymorphie und dynamischer Methodenwahl jederzeit bei einem Objekt vom Typ Sub ankommen kann, darf die überschreibende Methode nichts zurückliefern, was die überschriebene nicht auch hätte liefern können. Logisch möglich sind also die Varianten a) und b). Überschreiben mit kontravariantem Rückgabetyp ist logisch nicht zulässig, denn

```
Super s = new Sub();  
Tier t = s.m();
```

würde zu einem Typfehler führen, das die Variable t die gelieferte Referenz auf ein Object nicht aufnehmen kann.

2. Wenn es nun in der Klasse Super eine Methode m() gibt...

Klasse Super void m(Tier t) { ... }

und die Klasse Sub diese Methode überschreiben will, was ist dann rein logisch zulässig?

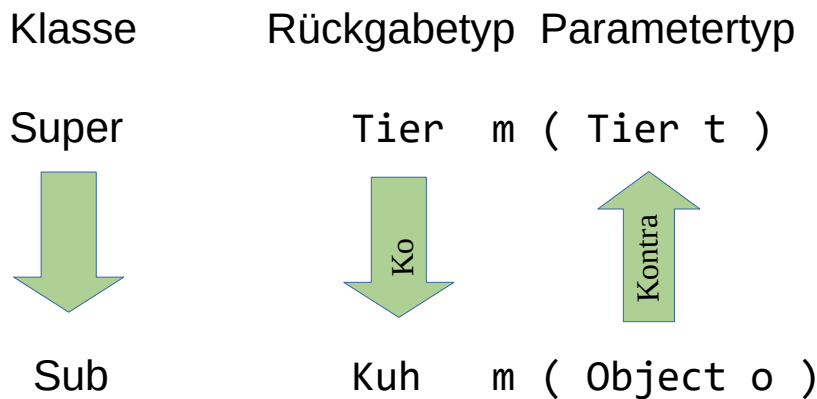
- a) Klasse Sub void m(Kuh k) { ... } (kovarianter Parametertyp)
- b) Klasse Sub void m(Tier t) { ... } (novarianter Parametertyp)
- c) Klasse Sub void m(Object o) { ... } (kontravarianter Parametertyp)

Da eine Nachricht, die an einen Ausdruck vom Typ Super gesendet wird, aufgrund von Polymorphie und dynamischer Methodenwahl jederzeit bei einem Objekt vom Typ Sub ankommen kann, muss die überschreibende Methode mindestens die Parameter „verkräften“, die die überschriebene verkräftet. Logisch möglich sind also die Varianten b) und c). Überschreiben mit kovariantem Parametertyp ist logisch nicht zulässig, denn

```
Super s = new Sub();  
s.m(new Tier());
```

würde zu einem Typfehler führen, da die Methode m() eines Sub das übergebene Tier nicht als Parameter „verkräftet“.

Die logisch möglichen Beziehungen beim Überschreiben lassen sich also folgendermaßen veranschaulichen, wobei der Pfeil jeweils den „Schritt“ vom Supertyp zum Subtyp symbolisiert: Wenn wir „vom Supertyp zum Subtyp gehen“, müssen wir bzgl. einer überschreibenden Methode beim Rückgabetyt „in die gleiche Richtung gehen“ (deshalb Kovarianz), bei den Parametertypen „in die entgegengesetzte Richtung gehen“ (deshalb Kontravarianz).



Zusätzlich ist bzgl. Rückgabetyt und Parametertypen auch Novarianz logisch möglich.

„Logisch möglich“ bedeutet hier: logisch möglich, unter der Voraussetzung, dass das LSP gilt („Überall dort, wo ein Objekt vom Typ Super stehen kann, ist auch ein Objekt vom Typ Sub erlaubt“), und dass Aufrufe von m() dynamisch gebunden werden.

„Logisch möglich“ heißt nicht, dass die jeweilige Möglichkeit in einer konkreten Programmiersprache auch so umgesetzt sein muss. Z.B. erlaubt Java beim Überschreiben zwar Kovarianz bzgl. des Rückgabetyps, bzgl. der Parametertypen ist hingegen nur Novarianz erlaubt. **!**

Und: „Logisch möglich“ bedeutet auch nicht unbedingt „sinnvoll“: Überschreiben mit kontravariantem Parametertyp ist im obigen Sinne logisch möglich, aber *nicht* sinnvoll: In der Realität haben speziellere Typen auch speziellere Methoden und zwar sowohl bzgl. des Rückgabetyps als auch bzgl. der Eingangsparameter! Beispiel: Wenn Tiere „Nahrung“ fressen, fressen spezielle Typen speziellere Nahrung, nicht allgemeinere. Insofern ist mit „Überschreiben mit kontravariantem Parametertyp“ leider nichts Sinnvolles anzufangen.

Will man Sachverhalte wie z.B. „Kühe sind spezielle Tiere, die eine spezielle Art Nahrung fressen, nämlich Gras“ modellieren kann man das in Java und ähnlichen Sprachen mit den Mitteln der parametrischen Polymorphie erreichen, wobei der Preis dafür eine Einschränkung der Subtypbeziehung ist. So ist in Java z.B. der Typ Tier<Nahrung> *nicht* Supertyp des Typs Tier<Gras>, auch wenn Gras Subtyp von Nahrung ist.

Darüber hinausgehend gibt es OO-Sprachen (z.B. Eiffel), die tatsächlich „Überschreiben mit kovarianten Parametertypen“ erlauben. Das steht im Gegensatz zu all dem oben Gesagten und entsprechend ist dafür dann auch wieder ein Preis zu bezahlen, nämlich Einschränkungen bzgl. der Polymorphie bei den beteiligten Typen und/oder bzgl. des dynamischen Bindens von auf solche Weise überschriebenen Methoden.